

Deep Space Planetary System Analysis

CSE 222/505 Homework 4

Computer Engineering Department, Gebze Technical University

Veysel Cemaloğlu, 200104004107

1. Introduction

This report documents the design and implementation of a tree-based data structure for a planetary system analysis application. The assignment required the development of a Java application that organizes celestial bodies (stars, planets, and moons) in a hierarchical tree structure, with the ability to process various commands for creating, querying, and analyzing the planetary system.

2. Problem Definition

The project involves creating a data structure that represents a planetary system, where:

- A star is the root of the tree structure
- Planets form a hierarchical chain with one planet per parent
- Multiple moons/satellites can be attached to a planet
- Each celestial body has attributes including name, type, and sensor data (temperature, pressure, humidity, radiation)

The application must support a command-line interface with various commands for managing and analyzing the planetary system, including creating a system, adding planets and satellites, finding radiation anomalies, path finding, and generating reports.

3. Approach and Architecture

3.1 Data Structures Used

The primary data structure used in this solution is a **tree**:

- Each node in the tree represents a celestial body (star, planet, or moon)
- The tree has a hierarchical structure with parent-child relationships
- Recursive Depth-First Search (DFS) traversal algorithms are used for operations on the tree

Additionally, a **Stack** is used for the getPathTo functionality, as specified in the requirements.

3.2 Class Design

The solution is organized into three main classes:

1. **DeepSpacePlanetarySystem**: Main application class that handles the command-line interface and processes user commands. It contains methods for:
 - Processing user input and command parsing
 - Creating and managing the planetary system
 - Traversing the tree structure using recursive DFS algorithms
 - Error handling and validation
2. **SpaceNode**: Represents a node in the tree, corresponding to a celestial body. It contains:
 - Basic properties (name, type)
 - References to sensor data
 - List of child nodes
 - Methods for managing children
3. **SensorData**: Contains sensor measurements for each celestial body:
 - Temperature (in Kelvin)
 - Pressure (in Pascals)
 - Humidity (percentage, 0-100)
 - Radiation (in Sieverts)

4. Implementation Details

4.1 Tree Structure Implementation

The tree structure is implemented using a node-based approach, where each SpaceNode contains:

- A reference to its sensor data
- A list of child nodes

The hierarchical rules enforced in the implementation include:

- The root must be a star
- A parent node (star or planet) can have at most one planet child
- A planet can have multiple moon/satellite children
- Moons cannot have children

4.2 Key Algorithms

4.2.1 Finding Nodes by Name

A recursive DFS algorithm is used to find nodes by name:

```
private SpaceNode findNode(SpaceNode node, String name) {  
    if (node.getName().equals(name)) {  
        return node;  
    }  
  
    for (SpaceNode child : node.getChildren()) {  
        SpaceNode result = findNode(child, name);  
        if (result != null) {  
            return result;  
        }  
    }  
  
    return null;  
}
```

This algorithm traverses the tree depth-first, checking each node's name and recursively searching children until the target node is found or the entire tree has been traversed.

4.2.2 Finding Radiation Anomalies

The findRadiationAnomalies command uses recursive DFS to identify nodes with radiation values exceeding a threshold:

```
private void findRadiationAnomaliesRecursive(SpaceNode node, double threshold,  
List<SpaceNode> anomalies) {  
    if (node.getSensorData().getRadiation() > threshold) {  
        anomalies.add(node);  
    }  
  
    for (SpaceNode child : node.getChildren()) {  
        findRadiationAnomaliesRecursive(child, threshold, anomalies);  
    }  
}
```

This method traverses the entire tree, checking each node against the threshold and adding matching nodes to a list.

4.2.3 Path Finding

The `getPathTo` command uses a stack to store the path from root to target:

```
private boolean getPathToRecursive(SpaceNode node, String targetName, Stack<String> path) {  
    if (node.getName().equals(targetName)) {  
        path.push(node.getName());  
        return true;  
    }  
  
    for (SpaceNode child : node.getChildren()) {  
        if (getPathToRecursive(child, targetName, path)) {  
            path.push(node.getName());  
            return true;  
        }  
    }  
  
    return false;  
}
```

This algorithm uses recursive DFS to find the target node, and then builds a path by pushing node names onto a stack as the recursion unwinds.

4.3 Error Handling

The implementation includes comprehensive error handling for various scenarios:

- Invalid parameter values (negative temperature, pressure, etc.)
- Duplicate node names
- Invalid parent-child relationships (e.g., trying to add a planet to a moon)
- Attempting to create multiple planetary systems
- Referencing non-existent nodes

Each error is handled with appropriate error messages to guide the user.

5. Testing Strategy

The solution was tested using a comprehensive test file containing commands that:

1. Test normal operation with valid inputs
2. Test error handling by providing invalid inputs
3. Test the hierarchical structure rules
4. Test all query commands with various parameters

The test cases cover:

- Creating a planetary system
- Adding planets in a hierarchical chain
- Adding multiple satellites to planets
- Attempting to violate structural rules (adding second planets, adding to invalid parents)
- Querying radiation anomalies with different thresholds
- Finding paths to various nodes
- Generating reports for the entire system and specific nodes

6. Challenges and Solutions

6.1 Hierarchical Structure Enforcement

Challenge: Ensuring that each parent node can have at most one planet child while allowing multiple satellites.

Solution: In the addPlanet method, a check was added to verify if the parent already has a planet child:

```
boolean hasPlaneChild = false;
```

```
for (SpaceNode child : parentNode.getChildren()) {
```

```
    if (child.getType().equals("Planet")) {
```

```
        hasPlaneChild = true;
```

```
        break;
```

```
    }
```

```
}
```

```
if (hasPlaneChild) {
```

```
    System.out.println("Error: Parent node already has a planet child.");
```

```
    return;
```

```
}
```

6.2 Tree Traversal and Formatting

Challenge: Printing the tree structure with proper indentation and formatting.

Solution: A recursive approach was used with prefix tracking to maintain proper indentation:

```
private void printMissionReportRecursive(SpaceNode node, String prefix, boolean isTail,
boolean isRoot) {
    if (isRoot) {
        System.out.println("└─ " + node.toString());
    } else {
        System.out.println(prefix + (isTail ? "└─ " : "└─ ") + node.toString());
    }

    List<SpaceNode> children = node.getChildren();
    for (int i = 0; i < children.size(); i++) {
        boolean lastChild = (i == children.size() - 1);
        String newPrefix = prefix + (isRoot ? "└─ " : (isTail ? "└─ " : "└─ "));
        printMissionReportRecursive(children.get(i), newPrefix, lastChild, false);
    }
}
```

6.3 Parameter Parsing

Challenge: Parsing command parameters with quoted strings.

Solution: A custom parsing method was implemented to handle quoted strings and space-separated values:

```
private String[] parseParameters(String args) {
    List<String> params = new ArrayList<>();
    StringBuilder currentParam = new StringBuilder();
    boolean inQuotes = false;

    for (int i = 0; i < args.length(); i++) {
        char c = args.charAt(i);
```

```

    if (c == '"') {
        inQuotes = !inQuotes;
    } else if (c == ' ' && !inQuotes) {
        if (currentParam.length() > 0) {
            params.add(currentParam.toString());
            currentParam = new StringBuilder();
        }
    } else {
        currentParam.append(c);
    }
}

if (currentParam.length() > 0) {
    params.add(currentParam.toString());
}

return params.toArray(new String[0]);
}

```

7. Conclusion

The Deep Space Planetary System Analysis application successfully implements a tree-based data structure for organizing and analyzing celestial bodies. The implementation meets all the requirements specified in the assignment, including:

- Command-line interface for user interaction
- Creation and management of a hierarchical planetary system
- Support for finding radiation anomalies
- Path finding with a stack data structure
- Mission report generation

The solution employs object-oriented design principles, recursive algorithms for tree traversal, and comprehensive error handling. The hierarchical structure of the tree accurately models the relationships between celestial bodies in a planetary system, with stars at the root, planets forming a chain, and moons as leaf nodes.